Xuleta — Qualitat de l'Aire (Vanilla JS)

Estructura MVC · ES6 Modules · Vanilla JS

Estructura de fitxers (MVC)

```
aireclar/
├── index.html
├── app.js                  ← Controlador + IIFE
├── model/
│   └── AirQuality.js       ← Model (dades)
├── service/
│   └── AirQualityService.js← Servei (AJAX)
├── view/
│   └── AirQualityView.js   ← Vista (DOM)
└── assets/
    ├── css/style.css
    └── js/dates.js         ← Utilitats dates
```

Exercici 1 — Carregar dates (1 punt)

Omplir el <select multiple> amb 365 dates usant getDates() i getSelectValues().

assets/js/dates.js — Afegir export a les funcions del professor:

```
export function getDates() {
    const dates = [];
    const start = new Date('2025-01-01T00:00:00Z');
    for (let i = 0; i < 365; i++) {
        const d = new Date(start);
        d.setUTCDate(start.getUTCDate() + i);
        dates.push(d.toISOString().slice(0, 10)); // 'YYYY-MM-DD'
    }
    return dates;
}

export function getSelectValues(selectNode) {
    return Array.from(selectNode.selectedOptions).map(opt => opt.value);
}
```

app.js — Importar i omplir el select:

```
import { getDates, getSelectValues } from './assets/js/dates.js';

const datesSelect = document.getElementById('datesSelect');
getDates().forEach(date => {
    const opt = document.createElement('option');
    opt.value = date;
    opt.textContent = date;
    datesSelect.appendChild(opt);
});
```

Exercici 2 — AJAX (1 punt)

Connectar a l'API Open-Meteo i obtenir pm10, pm2_5 i european_aqi per una data.

model/AirQuality.js — Classe MODEL:

```
export default class AirQuality {
    constructor(time, pm10, pm2_5, european_aqi) {
        this.time = time;
        this.pm10 = pm10;
        this.pm2_5 = pm2_5;
        this.european_aqi = european_aqi;
    }
}
```

service/AirQualityService.js — Classe SERVEI (fetch a l'API):

```
import AirQuality from '../model/AirQuality.js';

export class AirQualityService {
    static async cerca(date, latitude, longitude) {
        const url = `https://air-quality-api.open-meteo.com/v1/air-quality
            ?latitude=${latitude}&longitude=${longitude}
            &hourly=pm10,pm2_5,european_aqi
            &start_date=${date}&end_date=${date}`;

        const res = await fetch(url);
        if (!res.ok) throw new Error(`Error HTTP: ${res.status}`);
        const data = await res.json();

        const { time, pm10, pm2_5, european_aqi } = data.hourly;
        // data.hourly té arrays paral·lels: posició i = mateixa hora
        return time.map((t, i) => new AirQuality(t, pm10[i], pm2_5[i],
european_aqi[i]));
    }
}
```

view/AirQualityView.js — Mètode pintarFiles():

```
pintarFiles(records) {
    this.tbody.innerHTML = '';
    if (records.length === 0) {
        this.tbody.innerHTML = '<tr><td colspan="4">Sense resultats</td></tr>';
        return;
    }
    for (const r of records) {
        const tr = document.createElement('tr');
        tr.innerHTML = `
            <td>${r.time}</td>
            <td>${r.pm2_5 ?? '—'}</td>
            <td>${r.pm10 ?? '—'}</td>
            <td>${r.european_aqi ?? '—'}</td>
        `;
        this.tbody.appendChild(tr);
    }
}
```

Exercici 3 — Múltiples peticions (2 punts)

Permetre seleccionar N dates i combinar tots els resultats en un sol array.

app.js — Promise.all() per fer totes les peticions en paral·lel:

```
// Dins el listener del botó 'searchBtn':
```

```
const selectedDates = getSelectValues(datesSelect);

// Promise.all llança TOTES les peticions alhora (no una rere l'altra)
const results = await Promise.all(
    selectedDates.map(date => AirQualityService.cerca(date, latitude,
longitude))
);

// results = [ [AirQuality,...], [AirQuality,...], ... ] — array d'arrays
// .flat() ho converteix en un sol array pla
allData = results.flat();

// Guardem la primera data per al gràfic (Ex7)
firstDateRecords = results[0] || [];
```

Exercici 4 — Paginació (2 punts)

a) 5 resultats per pàgina amb botons Anterior/Següent. b) Amagar botons als extrems.

app.js — Lògica de paginació:

```
const RECORDS_PER_PAGE = 5;
let allData = [];
let currentPage = 1;

function numPages() {
    return Math.ceil(allData.length / RECORDS_PER_PAGE);
}

function changePage(page) {
    if (page < 1) page = 1;
    if (page > numPages()) page = numPages();

    const start = (page - 1) * RECORDS_PER_PAGE; // ex: pàg 2 → start=5
    const end   = page * RECORDS_PER_PAGE;        // ex: pàg 2 → end=10
    const pagData = allData.slice(start, end);    // agafam els 5 registres

    view.pintarFiles(pagData);
    view.actualitzarPaginacio(page, numPages());
}

document.getElementById('prevBtn').addEventListener('click', () => {
    if (currentPage > 1) { currentPage--; changePage(currentPage); }
});
document.getElementById('nextBtn').addEventListener('click', () => {
    if (currentPage < numPages()) { currentPage++; changePage(currentPage); }
});
```

view/AirQualityView.js — Amagar botons als extrems (part b):

```
actualitzarPaginacio(pagina, totalPagines) {
    this.pageInfo.textContent = `Pàgina ${pagina} de ${totalPagines || 1}`;

    // Amagar ANTERIOR si estem a la primera pàgina
    this.prevBtn.style.visibility = pagina <= 1 ? 'hidden' : 'visible';

    // Amagar SEGÜENT si estem a l'última o no hi ha dades
    this.nextBtn.style.visibility =
        (pagina >= totalPagines || totalPagines === 0) ? 'hidden' : 'visible';
}
```

Exercici 5 — API Geolocation (1,5 punts)

Usar navigator.geolocation per obtenir lat/lng reals i passar-les a l'API.

app.js — Geolocation amb fallback per defecte:

```
// Coordenades per defecte (s'usen si l'usuari denega el permís)
let latitude  = 39.5696;
let longitude = 2.6502;

if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(
        pos => {
            // Èxit: actualitzem amb les coordenades reals
            latitude  = pos.coords.latitude;
            longitude = pos.coords.longitude;
            view.mostrarMissatge(`Ubicació: ${latitude.toFixed(4)},
${longitude.toFixed(4)}`);
        },
        () => {
            // Error o denegat: les variables per defecte segueixen actives
            view.mostrarMissatge('Ubicació no disponible. Usant coordenades per
defecte.');
        }
    );
}

// Quan l'usuari prem Cerca, latitude i longitude ja tenen el valor correcte:
AirQualityService.cerca(date, latitude, longitude)
```

Exercici 6 — IIFE (1 punt)

Embolcallar tot el codi d'inicialització en una IIFE (Immediately Invoked Function Expression).

app.js — Estructura IIFE asíncrona:

```
// IIFE = funció que es defineix i s'executa immediatament
// El 'async' permet usar 'await' dins
(async () => {

    // Tot el codi va aquí dins:
    // - declaració de variables d'estat
    // - inicialització del select de dates
    // - geolocation
    // - listeners dels botons
    // - funcions changePage, numPages...

})(); // <-- els () del final fan que s'executi immediatament

// Avantatge: totes les variables (allData, currentPage...)
// queden dins el scope de la IIFE, sense contaminar el global.
```

Exercici 7 — Gràfic Chart.js (1,5 punts)

Gràfic de línies amb PM10, PM2.5 i AQI per hora, de la primera data seleccionada.

index.html — Carregar Chart.js i afegir el canvas:

```
<!-- Carregar la llibreria via CDN (a <head> o abans de </body>) -->
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<!-- Canvas on es dibuixarà el gràfic -->
<canvas id="chartCanvas"></canvas>
```

view/AirQualityView.js — Mètode pintarGrafic():

```
pintarGrafic(records) {
    if (!this.chartCanvas) return; // seguretat: canvas existeix?

    // Destruir gràfic anterior (evita errors si l'usuari cerca de nou)
    if (this._chart) this._chart.destroy();

    // Eix X: agafam 'HH:MM' de '2025-03-09T14:00' → slice(11,16) = '14:00'
    const labels = records.map(r => r.time.slice(11, 16));
    const pm10   = records.map(r => r.pm10);
    const pm2_5  = records.map(r => r.pm2_5);
    const aqi    = records.map(r => r.european_aqi);

    this._chart = new Chart(this.chartCanvas, {
        type: 'line',  // gràfic de línies
        data: {
            labels,    // eix X (hores)
            datasets: [
                { label: 'PM10',         data: pm10,  borderColor: '#e67e22',
                  tension: 0.3, pointRadius: 2 },
                { label: 'PM2.5',        data: pm2_5, borderColor: '#2980b9',
                  tension: 0.3, pointRadius: 2 },
                { label: 'European AQI', data: aqi,   borderColor: '#27ae60',
                  tension: 0.3, pointRadius: 2 },
            ]
        },
        options: {
            responsive: true,                       // s'adapta a la mida
            plugins: { legend: { position: 'top' } }, // llegenda a dalt
            scales: { y: { beginAtZero: true } }   // eix Y des de 0
        }
    });
}
```

app.js — Cridar el gràfic després de les peticions:

```
// results[0] = array de la primera data seleccionada
firstDateRecords = results[0] || [];
view.pintarGrafic(firstDateRecords);
```

Resum del flux complet

Quan l'usuari prem 'Cerca':

```
1. getSelectValues()      → array de dates seleccionades
2. Promise.all()          → fetch a l'API per cada data (en paral·lel)
3. results.flat()         → un sol array amb tots els registres
4. changePage(1)          → view.pintarFiles() amb els 5 primers
5. actualitzarPaginacio() → amagar/mostrar botons Anterior/Següent
6. view.pintarGrafic()    → gràfic Chart.js amb la primera data
```

— fi de la xuleta —